

RAPPORT D'INGÉNIERIE LOGICIELLE

Cycle de Vie Complet du Fichier XML BPMN 2.0 (Flowable → Backend → BDD → Frontend)

Projet : IncidentFlow V2

Auteur : Anas Benhammou

Composant : Flowable BPMN Engine & Reactive Sync



Étape 1 Conception Visuelle & Génération du XML BPMN 2.0

L'analyste métier ou le concepteur se connecte à la suite **Flowable Modeler** (<http://localhost:8082/flowable-ui/>). Il conçoit graphiquement les événements et tâches de traitement. Lors de l'enregistrement, l'outil sérialise le dessin sous la forme d'un document formel **XML standardisé OMG BPMN 2.0** :

```
←!— Extrait du fichier incident_workflow_v2.bpmn20.xml généré —→
<process id="incidentWorkflow" name="Incident Standard Workflow V2" isExecutable="true">
  <startEvent id="startEvent1" name="Création Incident"></startEvent>
  <userTask id="taskNouveau" name="Nouveau"></userTask>
  <userTask id="taskAssigne" name="Assigné"></userTask>
  <userTask id="taskEnCours" name="En cours"></userTask>
  <userTask id="taskResolu" name="Résolu"></userTask>
  <endEvent id="endEvent1" name="Clôture"></endEvent>
  <sequenceFlow id="flow4" sourceRef="taskEnCours" targetRef="taskResolu"></sequenceFlow>
</process>
```

Étape 2 Ingestion et Déploiement dans le Backend (Spring Boot)

Le fichier XML entre dans le Backend Java par deux canaux équivalents :

- **Canal A (Statique au démarrage)** : Placé dans backend/src/main/resources/processes/, le starter Spring Boot Flowable déploie automatiquement le fichier lors du bootstrap de l'application.
- **Canal B (Dynamique à chaud / Hot-Reload)** : L'administrateur téléverse le fichier via le composant React de l'application, appelant le point de terminaison REST POST `/api/workflows/deploy-flowable`.

```
// WorkflowController.java : Réception du fichier et soumission au moteur Flowable
Deployment deployment = repositoryService.createDeployment()
    .addInputStream(resourceName, file.getInputStream())
    .name("Dynamic Deployment from UI")
    .deploy();
```

Étape 3 Impact Physique en Base de Données : Schéma `flowable`

Dès l'exécution de l'instruction `.deploy()`, Flowable persiste le modèle dans son schéma relationnel dédié `flowable` sans toucher aux tables applicatives :

Table PostgreSQL	Rôle et Contenu Enregistré	Requête SQL de Vérification
<code>flowable.act_re_deployment</code>	Trace le numéro unique du déploiement et la date précise d'ingestion.	<pre>SELECT id_, name_, deploy_time_ FROM flowable.act_re_deployment;</pre>
<code>flowable.act_ge_bytearray</code>	Stocke le contenu brut du fichier XML sous forme de tableau d'octets binaires (BYTES_).	<pre>SELECT name_, length(bytes_) FROM flowable.act_ge_bytearray;</pre>
<code>flowable.act_re_procdef</code>	Contient les métadonnées parsées : la clé unique (<code>incidentWorkflow</code>) et le numéro incrémenté de version (<code>version_ = 1, 2, 3...</code>).	<pre>SELECT id_, key_, version_, name_ FROM flowable.act_re_procdef;</pre>

Étape 4 Synchronisation Automatique avec le Schéma Métier `incidentflow`

Pour éviter le couplage statique tout en garantissant la fluidité du Kanban, la méthode `WorkflowService.syncWithFlowable()` analyse le modèle BPMN en mémoire (`BpmnModel`) et propage les modifications vers les tables de configuration métier :

```
// Parcours ordonné du graphe BPMN le long des SequenceFlow
BpmnModel bpmnModel = repositoryService.getBpmnModel(procDef.getId());
List<UserTask> tasks = bpmnModel.getMainProcess().findFlowElementsOfType(UserTask.class);
// Mise à jour atomique de la table incidentflow.workflow_states et purge du cache Redis
workflow.getStates().clear();
workflow.getStates().add(new WorkflowState("Nouveau", ...));
workflowRepository.save(workflow);
```

Preuve en base de données : Une simple requête `SELECT name, label FROM incidentflow.workflow_states;` confirme que l'état modifié (ex: *Résolu*) a été mis à jour instantanément.

Étape 5 Consommation Frontend : Kanban, Stepper & Schéma Animé

Lorsque l'utilisateur navigue sur l'application React (port 3001), le frontend appelle `GET /api/workflows`. Les données synchronisées pilotent 3 composants graphiques majeurs :

- **Le Tableau Kanban (`KanbanView.jsx`) :** Génère dynamiquement les colonnes et distribue les incidents selon leur état actuel calculé par Flowable.
- **La Fiche Incident (`IncidentDetailView.jsx`) :** Affiche le stepper horizontal ordonné (1. *Nouveau* → 2. *Assigné* → 3. *En cours* → 4. *Résolu* → 5. *Clôturé*), marquant en vert les étapes déjà validées.
- **L'Écran d'Orchestration (`WorkflowConfigView.jsx`) :** Alimente le canevas interactif **React Flow**, dessinant les blocs d'étapes et animant en temps réel les flux de transitions avec des particules en mouvement continu.

Lorsqu'un opérateur manipule un incident, le cycle complet s'exécute de façon transparente :

1. **Création** : `runtimeService.startProcessInstanceByKey("incidentWorkflow")` crée une instance vivante dans `flowable.act_ru_execution` et insère l'ID unique (`process_instance_id`) dans `incidentflow.incidents`.
2. **Avancement** : Lors du déplacement Kanban, le Backend appelle `taskService.complete(taskId)`. Flowable avance le jeton vers la tâche suivante et inscrit la trace immuable dans `flowable.act_hi_taskinst` (Audit Trail).

Cette section regroupe les questions d'ingénierie les plus pointues susceptibles d'être posées par l'encadrant ou les membres du jury, accompagnées de leurs réponses méthodologiques et argumentées.

Q1. Pourquoi intégrer un moteur BPMN lourd (Flowable) plutôt que de coder le workflow en dur en Java (State Pattern ou if/switch) ?

Réponse de soutien : Dans la V1, chaque ajout d'état ou de règle de transition nécessitait de modifier le code Java, de recompiler l'application et de redéployer le serveur (couplage fort). Avec Flowable, la logique métier est externalisée selon la norme internationale **OMG BPMN 2.0** :

- **Agilité Métier (Low-Code/No-Code)** : Un analyste métier peut modifier un délai ou ajouter une étape d'approbation dans Flowable Modeler sans solliciter un développeur.
- **Traçabilité & Audit Automatiques** : Flowable gère nativement le journal d'audit immuable (qui a validé quoi, quand, et en combien de minutes), dispensant l'équipe d'écrire et maintenir des tables d'historique complexes.
- **Gestion des Versions** : Le moteur gère nativement la coexistence de plusieurs versions du même processus sans interruption de service.

Q2. Où et sous quelle forme le fichier XML importé est-il réellement stocké dans la base PostgreSQL ?

Réponse de soutien : Flowable ne crée pas de fichiers sur le disque dur du serveur ; il persiste tout dans le schéma relationnel flowable :

- Le fichier XML physique brut est converti en tableau d'octets binaires (BLOB) et stocké dans la colonne `bytes_` de la table `flowable.act_ge_bytearray`.
- Le moteur parse ensuite le XML et extrait la définition du processus dans la table `flowable.act_re_procdef` (avec son identifiant unique, sa clé `incidentWorkflow` et son numéro incrémenté de version).
- Chaque opération d'ingestion est historisée avec son horodatage précis dans `flowable.act_re_deployment`.

Q3. Pourquoi n'y a-t-il pas de clé étrangère SQL physique directe entre votre table 'incidents' et les tables de Flowable ?

Réponse de soutien : C'est l'application rigoureuse du concept de **Clé Étrangère Conceptuelle (Soft Foreign Key)**, fondamental dans l'architecture microservices et le Domain-Driven Design (DDD) :

- La table métier `incidentflow.incidents` stocke uniquement l'identifiant sous forme de chaîne : `process_instance_id`.
- **Règle d'or d'ingénierie** : Poser une contrainte SQL dure (`FOREIGN KEY REFERENCES flowable.act_ru_execution`) vers les tables internes d'un framework tiers est un anti-pattern. Cela empêcherait l'archivage ou la purge des anciens logs d'audit, bloquerait les montées de version indépendantes de Flowable, et créerait un couplage fort inter-domaines.

Q4. Que se passe-t-il si un utilisateur met à jour le XML alors que des incidents sont déjà en cours dans l'ancienne version ?

Réponse de soutien : Flowable applique le mécanisme de **Versioning non-destructif** :

- Les incidents en cours restent verrouillés sur la version du schéma sous laquelle ils sont nés (ex: Version 1). Leurs transitions continuent de s'exécuter normalement sans aucune régression.
- Seuls les incidents déclarés postérieurement au déploiement empruntent la nouvelle Version 2.
- Pour migrer des dossiers en cours vers la nouvelle version, Flowable fournit l'API avancée `ProcessInstanceMigrationBuilder`, permettant d'exécuter des plans de migration contrôlés sans

perte de données.

Q5. Comment le Frontend React (Kanban, Stepper et Graphe Animé) réagit-il de façon dynamique aux évolutions du XML ?

Réponse de soutenance : Le Frontend ne contient aucune logique de transition codée en dur :

- Au montage, l'application interroge `GET /api/workflows`.
- Le Backend exécute `syncWithFlowable()` qui inspecte le graphe `BpmnModel` en direct et retourne la liste ordonnée des étapes le long des `SequenceFlow`.
- Le Kanban génère automatiquement ses colonnes, la fiche de détail alimente son stepper horizontal (*Nouveau* → *Assigné* → *En cours* → *Résolu* → *Clôturé*), et le canevas **React Flow** trace dynamiquement le diagramme avec des flux particuliers animés.